

Assignment1: Inheritance Types in JS

1. Single Inheritance

Description:

A child class inherits from exactly ONE parent class. The child gets all properties and methods of the parent and can override them or add its own. Here Dog extends Animal.

Code:

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }
  speak() { return `${this.name} says '${this.sound}'`; }
  breathe() { return `${this.name} breathes air.`; }
}

// Dog inherits from Animal (Single Inheritance)
class Dog extends Animal {
  constructor(name, breed) {
    super(name, 'Woof');
    this.breed = breed;
  }
  fetch() { return `${this.name} (${this.breed}) fetches the ball!`; }
  speak() { return `${this.name} barks: '${this.sound}'!`; }
}

const animal = new Animal('Cat', 'Meow');
const dog = new Dog('Rex', 'German Shepherd');
console.log(animal.speak());
console.log(dog.speak());
console.log(dog.breathe());
console.log(dog.fetch());
```

Output:

=== SINGLE INHERITANCE ===

Parent class (Animal):

Cat says 'Meow'
Cat breathes air.

Child class (Dog extends Animal):

Rex barks: 'Woof!'
Rex breathes air.
Rex (German Shepherd) fetches the ball!

instanceof Animal? true
instanceof Dog? true

2. Multilevel Inheritance

Description:

Classes form a chain A -> B -> C. Each level extends the one above. The deepest child inherits from ALL ancestors. Here: Person -> Employee -> Manager.

Code:

```
class Person {
  constructor(name, age) { this.name = name; this.age = age; }
  introduce() { return `Hi, I am ${this.name}, ${this.age} years old.`; }
  eat() { return `${this.name} is eating.`; }
}

// Level 1: Employee extends Person
class Employee extends Person {
  constructor(name, age, company, salary) {
    super(name, age);
    this.company = company;
    this.salary = salary;
  }
  work() { return `${this.name} works at ${this.company}.`; }
  getSalary() { return `Salary: ${this.salary}`; }
}

// Level 2: Manager extends Employee (multilevel chain)
class Manager extends Employee {
  constructor(name, age, company, salary, teamSize) {
    super(name, age, company, salary);
    this.teamSize = teamSize;
  }
  manage() { return `${this.name} manages ${this.teamSize} people.`; }
  introduce() { return `Hi, I am ${this.name}, Manager at ${this.company}.`; }
}

const manager = new Manager('Carol', 40, 'Infosys', 2000000, 15);
console.log(manager.introduce());
console.log(manager.eat());
console.log(manager.work());
console.log(manager.manage());
```

Output:

```
=== MULTILEVEL INHERITANCE ===
Chain: Person -> Employee -> Manager
```

Level 0 - Person:

```
Hi, I am Alice, 25 years old.
Alice is eating.
```

Level 1 - Employee (extends Person):

```
Hi, I am Bob, 30 years old.
```

Bob works at TCS.

Bob's salary: Rs.8,00,000

Level 2 - Manager (extends Employee):

Hi, I am Carol, Manager at Infosys.

Carol is eating.

Carol works at Infosys.

Carol's salary: Rs.20,00,000

Carol manages a team of 15 people at Infosys.

manager instanceof Person? true

manager instanceof Employee? true

manager instanceof Manager? true

3. Hierarchical Inheritance

Description:

ONE parent class is inherited by MULTIPLE child classes independently. Each child shares parent behaviour but can add its own. Here: Vehicle -> Car, Bike, Truck.

Code:

```
class Vehicle {
  constructor(brand, speed) { this.brand = brand; this.speed = speed; }
  move() { return `${this.brand} is moving at ${this.speed} km/h.`; }
  fuelType() { return `${this.brand} runs on fuel.`; }
}

class Car extends Vehicle {
  constructor(brand, speed, doors) { super(brand, speed); this.doors = doors; }
  honk() { return `${this.brand} car honks: Beep Beep!`; }
}

class Bike extends Vehicle {
  constructor(brand, speed, type) { super(brand, speed); this.type = type; }
  wheelie() { return `${this.brand} ${this.type} bike does a wheelie!`; }
}

class Truck extends Vehicle {
  constructor(brand, speed, payload) { super(brand, speed); this.payload = payload; }
  loadCargo() { return `${this.brand} truck loads ${this.payload} tons.`; }
}

const car = new Car('Toyota', 180, 4);
const bike = new Bike('Royal Enfield', 140, 'Cruiser');
const truck = new Truck('Tata', 100, 20);
console.log(car.move());
console.log(car.honk());
console.log(bike.move());
console.log(bike.wheelie());
console.log(truck.move());
console.log(truck.loadCargo());
```

Output:

```
=== HIERARCHICAL INHERITANCE ===
Parent: Vehicle | Children: Car, Bike, Truck
```

```
--- Car (extends Vehicle) ---
Car: Toyota, 4 doors, 180 km/h
Toyota is moving at 180 km/h.
Toyota runs on fuel.
Toyota car honks: Beep Beep!
```

```
--- Bike (extends Vehicle) ---
Bike: Royal Enfield (Cruiser), 140 km/h
```

Royal Enfield is moving at 140 km/h.
Royal Enfield Cruiser bike does a wheelie!

--- Truck (extends Vehicle) ---

Tata is moving at 100 km/h.
Tata truck loads 20 tons of cargo.

car instanceof Vehicle? true
bike instanceof Vehicle? true
car instanceof Bike? false

4. Multiple Inheritance (Mixin Pattern)

Description:

JavaScript does NOT natively support multiple inheritance. The Mixin pattern composes multiple behaviours onto one class using higher-order functions.

Code:

```
const Flyable = (Base) => class extends Base {
  fly() { return `${this.name} soars through the sky!`; }
};

const Swimmable = (Base) => class extends Base {
  swim() { return `${this.name} dives in and swims!`; }
};

const Runnable = (Base) => class extends Base {
  run() { return `${this.name} runs at full speed!`; }
};

class Animal {
  constructor(name, species) { this.name = name; this.species = species; }
  describe() { return `${this.name} is a ${this.species}.`; }
}

// Duck inherits from ALL THREE mixins + Animal
class Duck extends Runnable(Swimmable(Flyable(Animal))) {
  constructor(n) { super(n, 'Duck'); }
  quack() { return `${this.name} quacks: Quack Quack!`; }
}

const duck = new Duck('Donald');
console.log(duck.describe());
console.log(duck.fly());
console.log(duck.swim());
console.log(duck.run());
console.log(duck.quack());
```

Output:

```
=== MULTIPLE INHERITANCE (Mixin Pattern) ===
Mixins: Flyable, Swimmable, Runnable

--- Duck (Flyable + Swimmable + Runnable + Animal) ---
Donald is a Duck.
Donald soars through the sky!
Donald dives in and swims!
Donald runs at full speed!
Donald quacks: Quack Quack!

--- Eagle (Flyable + Runnable + Animal) ---
Bald Eagle soars through the sky!
```

```
Bald Eagle hunts prey from above!  
Can eagle swim? false
```

```
--- Fish (Swimmable + Animal) ---  
Nemo dives in and swims!  
Can fish fly? false
```

```
duck instanceof Animal? true  
'fly' in duck? true  
'swim' in duck? true
```

5. Hybrid Inheritance

Description:

Combination of two or more inheritance types. Here: Hierarchical (Shape->TwoDShape/ThreeDShape) + Multilevel (TwoDShape->Circle/Rectangle) + Mixin (Colorable applied to Sphere).

Code:

```
const Colorable = (Base) => class extends Base {
  setColor(c) { this.color = c; return `Color set to ${c}.`; }
  getColor() { return `Color: ${this.color || 'none'}`; }
};

class Shape { constructor(name) { this.name = name; } }

class TwoDShape extends Shape {
  info() { return `${this.name} | 2D | Area=${this.area().toFixed(2)}`; }
}

class ThreeDShape extends Shape {
  info() { return `${this.name} | 3D | Vol=${this.volume().toFixed(2)}`; }
}

class Circle extends TwoDShape {
  constructor(r) { super('Circle'); this.radius = r; }
  area() { return Math.PI * this.radius ** 2; }
}

class Rectangle extends TwoDShape {
  constructor(w, h) { super('Rectangle'); this.width=w; this.height=h; }
  area() { return this.width * this.height; }
}

// Sphere: ThreeDShape + Colorable mixin
class Sphere extends Colorable(ThreeDShape) {
  constructor(r, color) { super('Sphere'); this.radius=r; this.color=color; }
  area() { return 4 * Math.PI * this.radius ** 2; }
  volume() { return (4/3) * Math.PI * this.radius ** 3; }
}

const circle = new Circle(7);
const sphere = new Sphere(4, 'Blue');
console.log(circle.info());
console.log(sphere.info());
console.log(sphere.getColor());
console.log(sphere.setColor('Gold'));
```

Output:

```
=== HYBRID INHERITANCE ===
Combining: Hierarchical + Multilevel + Mixin

--- Circle (Shape -> TwoDShape -> Circle) ---
Circle | 2D | Area=153.94 | Perimeter=43.98
```

```
instanceof Shape?      true
instanceof TwoDShape? true

--- Rectangle (Shape -> TwoDShape -> Rectangle) ---
Rectangle | 2D | Area=50.00 | Perimeter=30.00
instanceof TwoDShape? true

--- Sphere (Shape -> ThreeDShape + Colorable) ---
Sphere | 3D | Area=201.06 | Volume=268.08
Color: Blue
Sphere color set to Gold.
instanceof ThreeDShape? true
'setColor' in sphere?    true
```